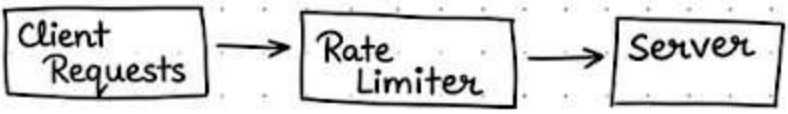
Lecture 5: API Rate Limiting & Create Your Own Rate Limiter

What is API Rate Limiting ?

- Rate Limiting ek technique hai joh control hai ek client ek specific time frame mein kitni request server ko bhej sakta hai...
- Jaise highway par limited cars hi pass to sakti hai, watse hi server par bhi limited request allow hoti hai - to avoid overload



Why is Rate Limiting important ?

1. Protection from Attacks
   - DDOS / DOS attacks se server down ho sakta hai
   - Rate Limiting acts as a shield, to protect the system.
2. Managing cost
   - Har request mein lagta hai CPU, Memory, Bandwidth
   - Example - Zomato, Swiggy, Google Maps = Millions of Api calls
   - Each request is expensive

Definition of Rate Limiting

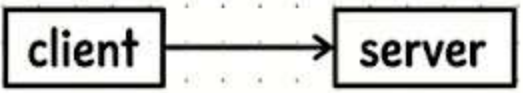
"EK Client ek time frame mein kitni requests bhej skta hai ?"
Iss limit ko enforce karta hai server yaa application - to ensure Stability and efficiency.

Ways to implement Rate Limiting

- There are three ways to implement Rate limiting.
  1. Client-Side Rate Limiting.
  2. Server-Side Rate Limiting.
  3. Middleware-Side Rate Limiting.

1. Client-Side Rate Limiting

- Client ko hi rate limits bataye jaate hain
- Client app / browser requests ko khud regulate karta hai



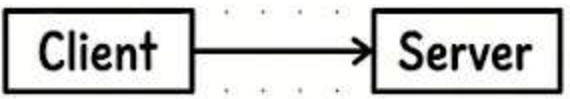
Client-Side Rate Limiting

Problems

1. Yeh methods unreliable hai
2. Can easily be bypassed coz its added in client code & can be changed by inspect in browser.
3. Server phir bhi vulnerable ho sakta hai

2. Server-Side Rate Limiting

- Server khud har client ka rate track karta hai
- Requests aane se pehle check hota hai:
  - Quota ke andaar → Process
  - Exceed → Block



Server-Side Rate Limiting

3. Middleware based Rate Limiting

Ek intermediary layer hoti hai

Client → middleware → Server

Middleware har request ko :

1. intercept karta hai
2. Rate limiting logic apply karta hai
3. Valid request ko forward Karta hai



Benefits

1. Centralized logic (server se alag)
2. Easily scalable
3. Extra layer of security before server

Which One Should we implement : server-side or middleware ?

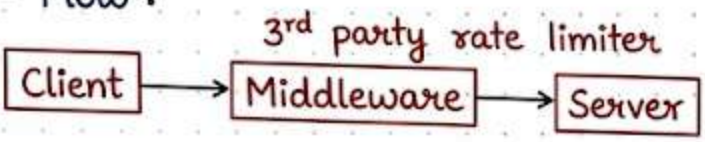
Yeh aapke application use case par depend karta hai -

1. Server-Side implementation
   - Aagar apki server-side language (Java/C++/JS/ python) hai → which is fast, toh it's better to do server-side par rate limiting implement kare.
   - Flow : Client → Server → Rate Limiting



2. Middleware implementation
   - Alternative way : 3rd party rate limiter providers ka use karenge like
     - Amazon AWS API Gateway.
   - yeh providers middleware layer ke upar rate limiting implement karte hai

- Flow :

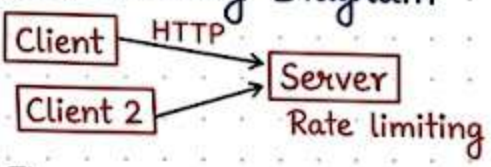


- We don't have to worry about - logic - Amazon will manage itself.

Rate limiting on which basis ?

- IP-based : Client ke IP address, ke basís par
- User-based : User-id ke basis par
- Other ways : Primary Key, candidate key, ke, basis par.

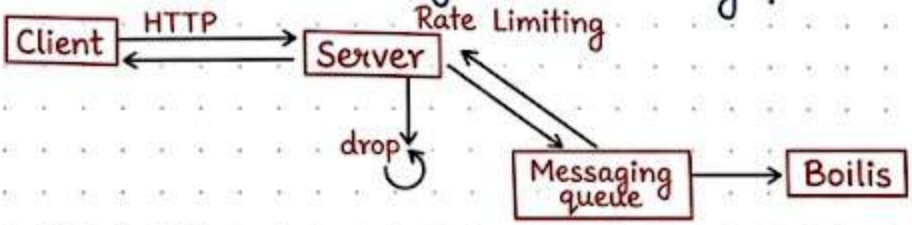
Rate Limiting Diagram



⊙ Flow

- Client → Rate Limiting → server → Message queue
- Server → drop (429) when limit ,, exceeds
  - Ex : 2 req / second on user ID exists

What happens during Rate Limiting ?



ex: 2 req / sec on the basis of user ID

Response Code

200 : ok
429 : Too many requests

Headers

x-api-limit ⩾ 2
x-duration > seconds

Important points

- dropping requests silently (without notifying clients) is not a good method : Not a good idea as client is unaware

Response code in Rate limiting

Code	Meaning
200	OK (Req. accepted)
429	Too many requests (Limit exceeded)

⊙ Server provider info deta hai like :

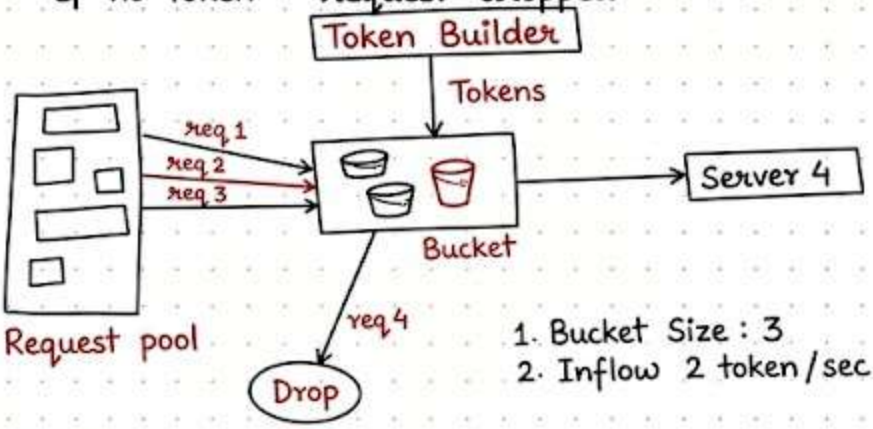
- x-api-limit : 2 (allowed requests)
- x-duration : time frame in seconds

Algorithms of Rate limiting

1. Token Bucket Algorithm

⊙ Requests (req 1, req 2, req 3, req 4) → Bucket (capacity 3 token) → Server

- Token Builder adds token to bucket at 2 token/sec
- If Bucket full → new token overflow (east)
- Requests uses 1 token to proceed.
- If no token → request dropped



pros

1. Simple to implement
2. Can handle short bursts of traffic

Con

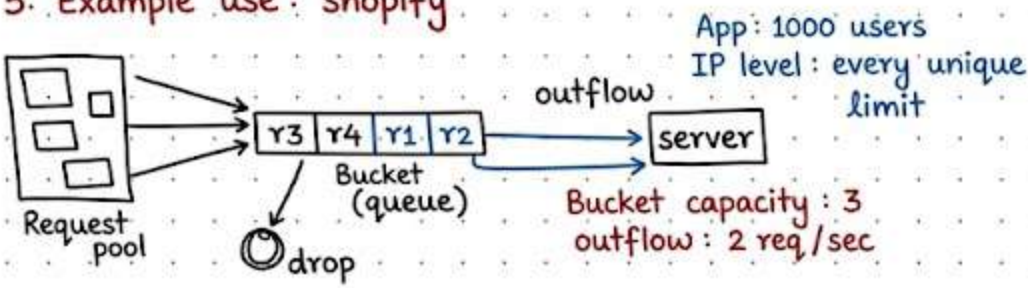
1. Requires accurate info to decide bucket size & token inflow rate

2. Leaky Bucket Algorithm (Global Rate Limiting)

Request (r1, r2, r3, r4) → Bucket (queue capacity 3) → server (handles outflow 2 req/sec)

key points

1. Bucket capacity : 3 requests
2. Outflow rate : 2 req/sec
3. Server processes requests at fixed outflow rate
4. Middleware throttles requests so server not overwhelmed.
5. Example use : shopify



⊙ pros

1. Simple to implement
2. Prevents server crash even during traffic bursts

⊙ Cons

1. During DDOS/DOS, server may still miss legitimate valuable requests.
2. Requests exceeding outflow get dropped.

Application Example

1000 APIs → API Gateway applies limiting → 2 bucket limit per API → handles overload.

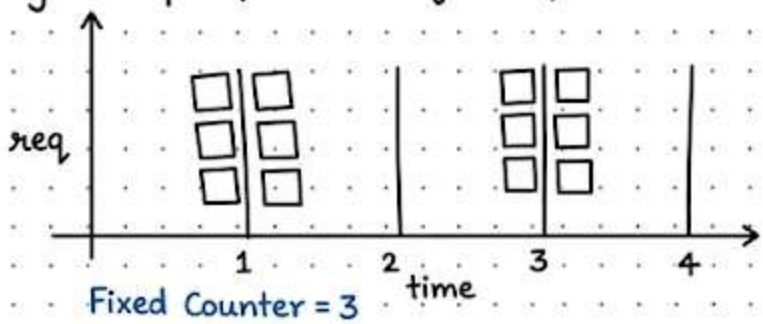
3. Fixed Window Counter

- In a fixed time interval (eg. 1 second or 1 minute), only a specific number are allowed

Diagram

Time blocks 1, 2, 3, 4 with requests shown im each, block 2 has 3 requests.

In a fixed interval (say a sec, or a minute). Only a specific no. of req can come.



Details

1. Fixed Counter limit : 3 requests per interval
2. Requests exceeding the limit in the window are dropped.
3. Can be applied based on IP, User Id, or Client Id

Cons

1. If burst traffic hits at the end of the window, it can cause server overload and high latency.
2. Examples : In a 3-minute window, if 6 requests come at the last second, all will be processed together, causing potential issues.

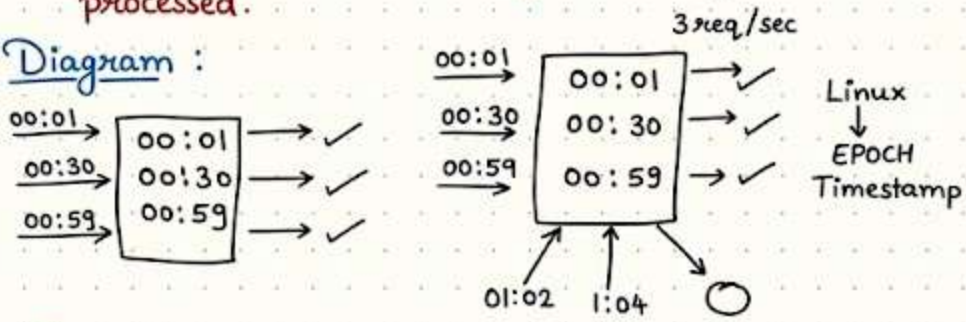
4. Sliding Window Log Algorithm

- Considered the best and strictest rate limiting algorithm.
- Tracks every request individually in a log file.

How it works ?

- Stores timestamp of each request in a log.
- On new requests:
  1. Remove outdated requests from the log (older than window duration)
  2. Adds new request to the log.
  3. Checks if request count exceeds the limit
  4. If limit exceeded → request dropped, else processed.

Diagram :



Example

- A request arriving at 01:01 (within 1-minute window) will be dropped if limit reached
- Sliding window counts requests in the last N minutes (eg. last 3 minutes)

Pros

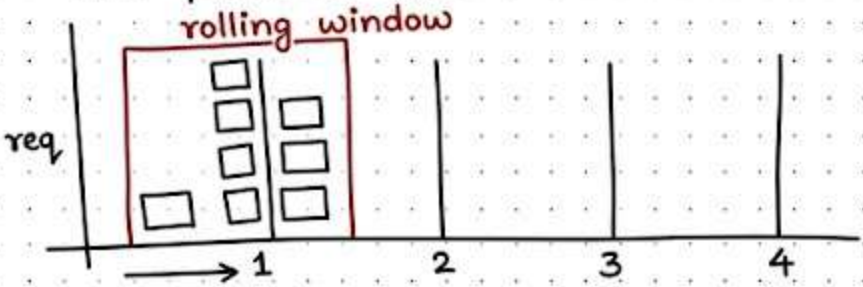
- Very strict enforcement of limits

Cons

- Slow due to frequent log updates.
- Memory-intensive to store request logs.

5. Sliding Window Counter Algorithm

- hybrid approach combining fixed window & sliding window Log algorithms
- counts requests in the current interval plus a weighted fraction of requests from the previous interval



Formula : No. of req in curr interval + No. of req. in prev Interval * 70%

$$= 4 + 5 \times 0.7$$
$$= 7.5$$

purpose

- provides smoother rate limiting by considering partial previous Internal traffic
- Avolds sudden spikes in requests count at window boundaries

Create Your Own Rate Limiter

Q] Which Algorithm to implement ?

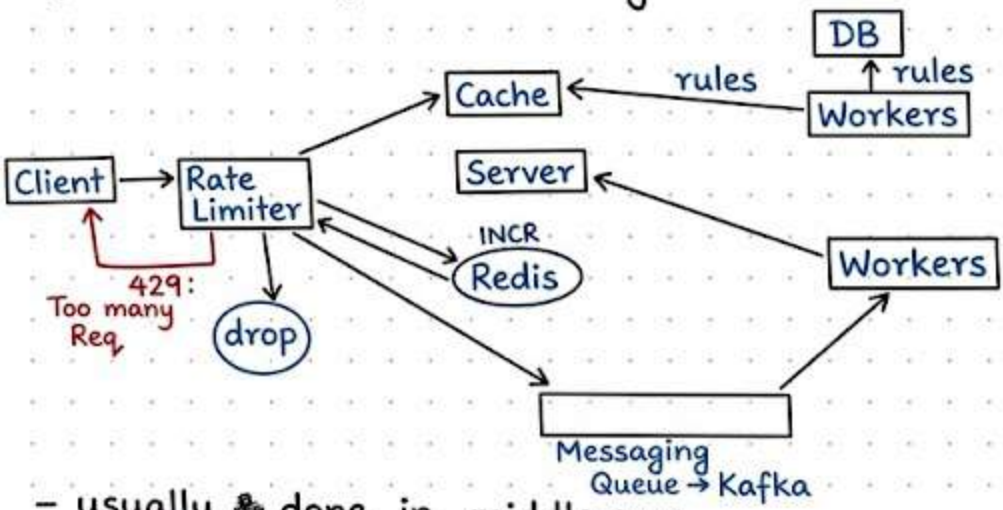
- Fixed Window Counter is a simple & common choice
- Examples : ~~erstrow~~

Time Block : 1 | 2 | 3 | 4
Expected Count : 3
Current Count : 3

Q) Storage for counters

- Counter Storage options
  - Database (db) : SQL / NoSQL (slow)
  - Cache : In-memory cache like Redis (fast)

Implementation of Rate Limiting



- usually & done in middleware
- middleware intercepts requests, check limits and allows or block accordingly.

@system Architecture

Components.

- DB - Stores rules and data (slow)
- Cache (Redis) - Stores counters and rules for quick access.
- Worker - periodically syncs rules from DB to cache.
- Rate limiter - uses cached counters to enforce limits.
- Messaging queue (Kafka) - Handles high priority requests & async processing

Client → Rate limiter → server flow

- Redis Logic
  - Counter example : If counter = 4 and limit exceeded, request denied.
  - Redis key expiry : 1 minute - after which counter resets to 0.
  - Cache reduces DB load by storing rules & counters.
  - Workers regularly fetch rules from DB & update cache.
  - High priority requests pushed to kafka for async handling.